

Software Engineering

Exam Suggestion — Complete Answer Guide
North Bengal International University | CSE Department

Q1. What is Software? What is Software Engineering?

Software

Software is a collection of computer programs, procedures, rules, and related documentation that tells a computer what to do. Unlike hardware, you cannot physically touch software — it exists as code and data.

Real-life examples:

- MS Word — a word processing program
- Google Chrome — a web browser
- Facebook App — a social networking application
- Operating Systems like Windows or Linux — control the entire computer

Software Engineering

Software Engineering is a systematic and disciplined approach to designing, developing, testing, and maintaining software. It applies engineering principles so that software is reliable, efficient, and easy to maintain.

IEEE Definition: "Software Engineering is the application of engineering principles to software development."

Objectives of Software Engineering

- Develop high-quality software that meets user needs
- Reduce development cost by using structured processes
- Complete projects on time by following proper planning
- Improve maintainability so future changes are easy
- Increase reliability so software works without failures

Q2. Software Doesn't Wear Out but It Does Deteriorate — Explain

Physical products like cars or machines wear out over time because parts break. Software has no physical parts, so it does not wear out. But software does deteriorate — its quality gradually decreases due to changes and modifications over time.

Why Hardware Wears Out

- Mechanical parts get damaged with constant use
- Failure rate is high at first, then drops, then rises again at end-of-life

Why Software Deteriorates

- Every time software is updated, new bugs may be introduced
- Adding new features makes the code more complex
- Poor documentation makes future maintenance harder

- Old code becomes incompatible with new technologies

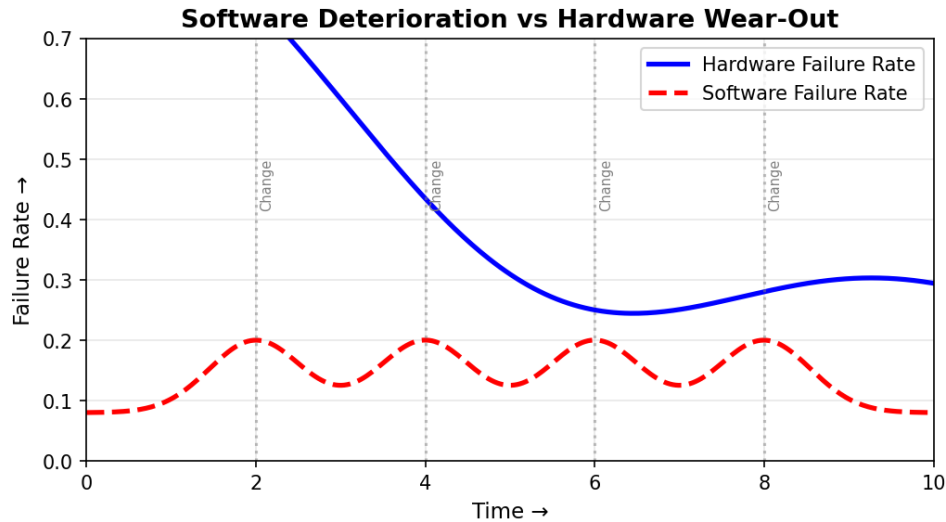


Figure: Hardware failure rate follows a bathtub curve. Software starts stable but deteriorates after repeated changes and modifications.

Conclusion

Software does not wear out physically. But poor maintenance and frequent modifications gradually reduce its quality — this is called software deterioration.

Q3. Software Applications — Types and Real-Life Examples

Software applications are programs designed to help users perform specific tasks. Different types of software serve different purposes.

Type of Software	Description	Real-Life Example
System Software	Controls hardware and manages resources	Windows, Linux, macOS
Business Software	Handles business operations and records	Payroll System, ERP Software
Web Applications	Runs on a browser through the internet	Facebook, Amazon, Gmail
Mobile Applications	Runs on smartphones and tablets	WhatsApp, Messenger, Maps
Scientific Software	Used for research and calculations	MATLAB, SPSS, AutoCAD
Embedded Software	Built into hardware devices	ATM Machine, Microwave Oven
Educational Software	Supports learning and teaching	Google Classroom, Duolingo

Real-Life Applications of Software

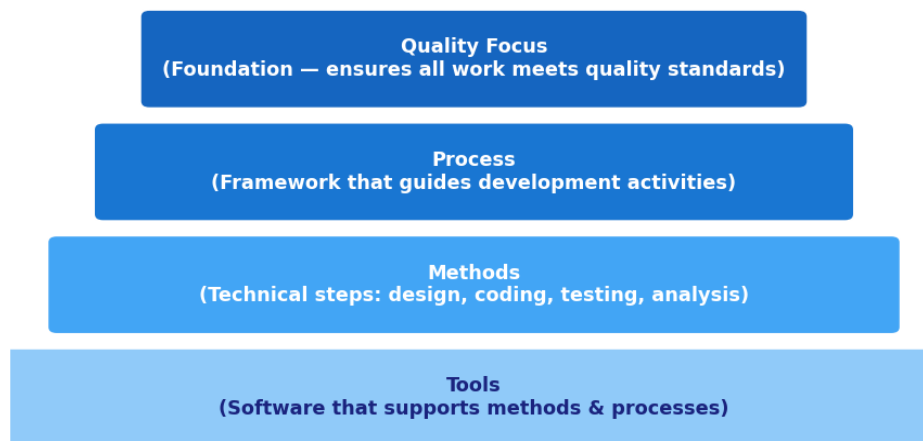
- Online Banking — customers check balances, transfer money, pay bills

- E-Commerce — buying and selling products online (Daraz, Amazon)
- Hospital Management — track patients, appointments, medicine records
- Railway Reservation — book tickets and check train schedules
- University Result System — publish and access student results
- Social Networking — connect and communicate with people globally

Q4. Layered Technology in Software Engineering

Software Engineering is built on four layers, like a pyramid. Each layer supports the one above it. Together they ensure that software is developed in a structured and high-quality way.

Software Engineering — Layered Technology



Layer 1 — Quality Focus (Foundation)

This is the base of everything. All software engineering activities aim to produce high-quality software. Without a focus on quality, even the best tools and methods will not produce good results.

Layer 2 — Process

The process layer provides a framework that organizes all development activities. It defines what needs to be done, when, and in what order. Examples include the Waterfall process and the Agile process.

Layer 3 — Methods

Methods are the technical procedures used during development. They cover how to analyze requirements, design the system, write code, and test it. Methods give engineers step-by-step guidance.

Layer 4 — Tools

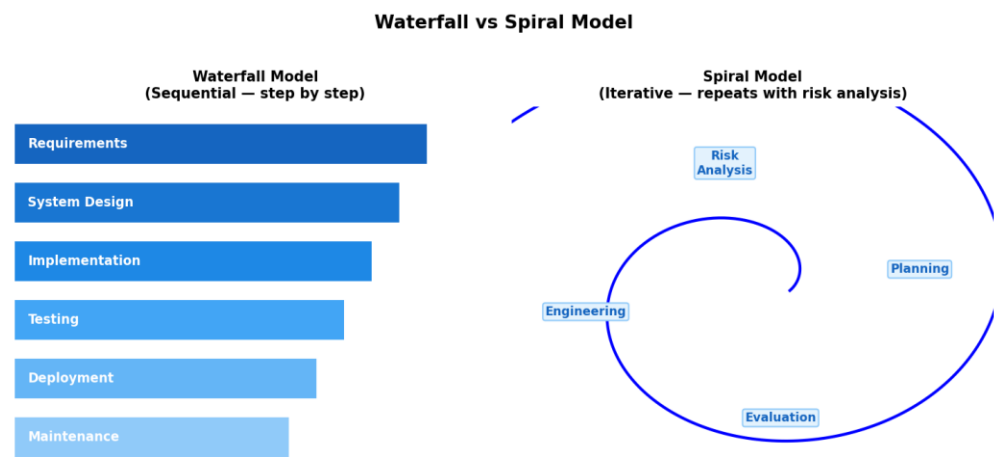
Tools are software applications that support the methods and processes. Examples: IDE tools like VS Code for coding, Git for version control, and testing tools like Selenium. Tools automate many tasks and improve efficiency.

Why is Layered Technology Important?

- Ensures software development is structured and organized
- Helps teams produce reliable and high-quality software
- Each layer builds on the one below, making everything connected

Q5. Waterfall Model vs Spiral Model — Comparison

Both are software development process models, but they work in completely different ways.



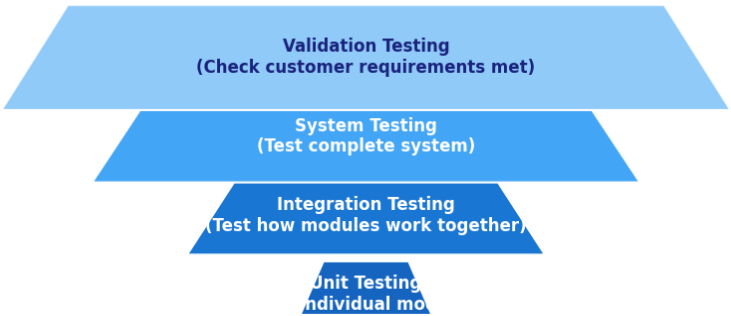
Feature	Waterfall Model	Spiral Model
Approach	Sequential — one phase at a time	Iterative — repeats in cycles
Risk Analysis	Not included	Included in every cycle
Flexibility	Low — hard to go back	High — easy to adjust
Customer Feedback	Only at end of project	Continuous at each cycle
Cost	Lower for simple projects	Higher due to repeated analysis
Suitable For	Small, well-defined projects	Large, complex, risky projects

Conclusion

Waterfall is simple and best when requirements are clear from the start. Spiral is better when there are many risks and requirements may change over time.

Q6. Software Testing — Methods and Strategies

Software Testing is the process of running software to find defects and verify that it works correctly. Testing helps improve the quality of software before it reaches users.



1. Unit Testing

Tests individual modules or functions in isolation. The goal is to check that each small piece of code works correctly on its own. Usually done by developers during coding.

2. Integration Testing

Tests how multiple modules work together when combined. There are two approaches:

- Top-Down Integration — start from high-level modules, add lower ones gradually
- Bottom-Up Integration — start from low-level modules, build upward

3. Validation Testing

Checks whether the software meets the customer's requirements. This answers the question: "Does the software do what the user wanted?"

4. System Testing

Tests the complete integrated software system as a whole. It checks performance, security, and overall behavior under real conditions.

Other Testing Methods

Testing Type	Focus	Who Does It
Black Box Testing	Tests input/output without looking at code	Testers (no code knowledge needed)
White Box Testing	Tests internal code logic and structure	Developers (code knowledge required)
Alpha Testing	Early testing before public release	Done by the development team
Beta Testing	Real-world testing before final release	Done by actual end users

Q7. What is SQA? Explain Its Role.

Software Quality Assurance (SQA) is a set of planned and systematic activities that ensure both the software development process and the final software product meet defined quality standards.

Simply put: SQA makes sure the team follows proper procedures so the final product is reliable and correct.

Roles of SQA

1. Establish quality standards and guidelines for the development team
2. Monitor the development process to ensure standards are followed
3. Conduct reviews, walkthroughs, and audits at each stage
4. Ensure compliance with company and industry procedures
5. Detect defects early — fixing early is cheaper than fixing later
6. Improve software reliability and overall product quality

Benefits of SQA

Benefit	Explanation
Better Software Quality	Products have fewer bugs and meet user expectations

Benefit	Explanation
Reduced Maintenance Cost	Fewer defects mean less time fixing problems later
Increased Customer Satisfaction	Users receive software that works correctly and reliably

Q8. What is DFD? Why is it Needed?

A Data Flow Diagram (DFD) is a graphical tool that shows how data moves through a software system. It uses simple symbols to represent processes, data storage, external users, and data flow directions.

Four Components of a DFD

Symbol	Name	Meaning
Circle / Oval	Process	Represents a function or activity that transforms data
Arrow (→)	Data Flow	Shows the direction data moves between components
Open Rectangle	Data Store	Represents where data is stored (database, file)
Rectangle	External Entity	People or systems that interact with the software

Why is DFD Needed?

- Helps understand how the system works from a data perspective
- Makes it easy to visualize data movement without writing code
- Improves communication between developers and non-technical stakeholders
- Assists in system analysis and design planning
- Helps identify inefficiencies and missing data flows early

Q9. Identify Data Flow in a Given DFD

In a DFD, a Data Flow represents the movement of data between system components. It is shown as a labeled arrow pointing from source to destination.

Symbol

→ (Arrow with label) Example: Student →[Registration Form]→ System

Common Data Flow Examples

Data Flow Name	Meaning
Student Information	Student details sent to the system
Book Request	Student asks for a book from the library
Result Data	Processed results sent to students

Data Flow Name	Meaning
Payment Information	Payment details sent from customer to system
Registration Form	Form filled by user and submitted to system

Key Rule: Every arrow in a DFD must have a label. The label describes what data is being transferred. The direction of the arrow shows where the data goes.

Q10. DFD — Library and Result Management System

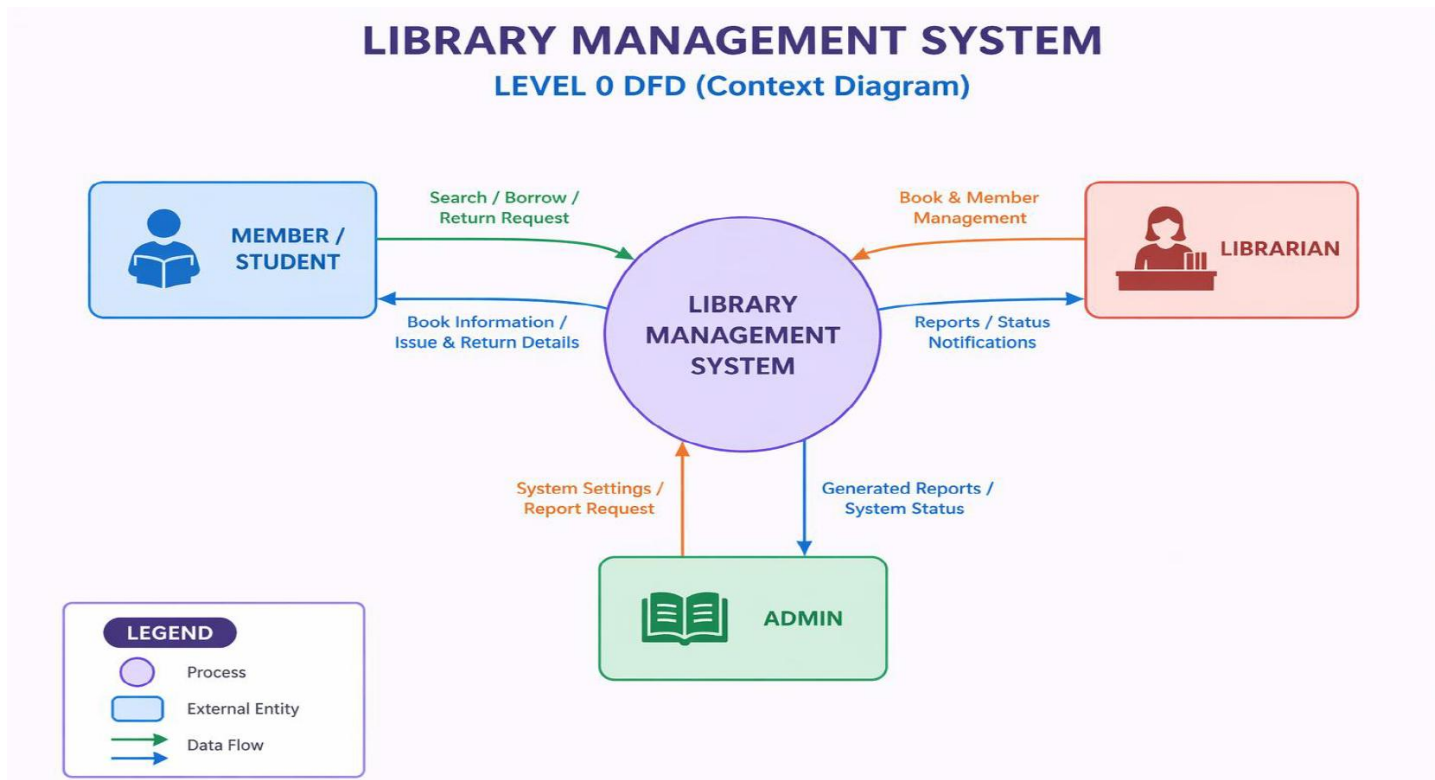
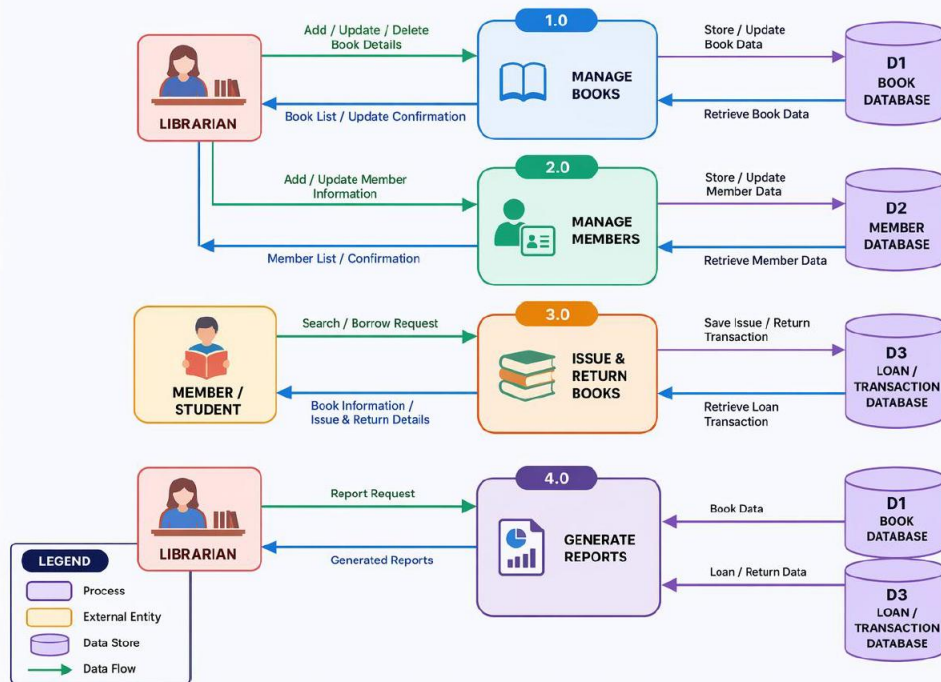


Figure: Level-0 DFD of a Library Management System showing data flows between Student, Admin, the system, and the Book Database.

Library Management System — Data Flows

LEVEL 1 DFD – LIBRARY MANAGEMENT SYSTEM

Decomposition of Library Management System (Level 0)



Result Management System — Data Flows

Data Flow	Direction	Description
Marks Entry	Teacher → System	Teacher submits student marks into the system
Result Processing	System (internal)	System calculates grades from entered marks
Result Publication	System → Student	Final results are displayed to students

Q11. What is Risk in Software Engineering? Types.

A Risk is a potential problem that might happen in the future and negatively affect a software project. Risks are uncertain — they may or may not occur — but if they do, they can cause delays, extra costs, or project failure.

Types of Risk

Type	What It Affects	Examples
Project Risk	Schedule, budget, or resources	Staff shortage, budget cuts, unclear requirements
Technical Risk	Software quality and functionality	New unproven technology, complex design, integration issues
Business Risk	Business success and market position	Market competition, customer rejection, product becoming outdated

Why Risk Management Matters

- Helps teams prepare for unexpected problems in advance
- Reduces the chance of project failure or budget overrun
- Allows teams to make better decisions under uncertainty

Q12. Reactive vs Proactive Risk Strategy

Reactive Risk Strategy

In a reactive strategy, the team takes action only after a risk has already happened. The team waits for problems to appear and then tries to fix them.

Example: A server crashes and then the team starts repairing it. No backup was prepared beforehand.

Disadvantage: It is expensive, slow, and often causes project delays.

Proactive Risk Strategy

In a proactive strategy, the team identifies risks before they happen and takes steps to prevent or reduce them. This is the better approach.

Example: Setting up a backup server before any crash happens. Security measures are implemented before any attack.

Advantage: Saves time and money. Prevents major disruptions to the project.

Feature	Reactive Strategy	Proactive Strategy
When Action is Taken	After the risk has occurred	Before the risk occurs
Approach	Problem-driven (react to problems)	Prevention-driven (prevent problems)
Cost	Higher — damage already done	Lower — prevention is cheaper
Project Impact	Can cause major delays	Minimizes impact on the project
Best Practice?	No — not recommended	Yes — always preferred

Q13. RMMM — Risk Mitigation, Monitoring, and Management

RMMM is a complete risk handling framework used in software engineering. It has three main activities that work together to handle risks throughout the project lifecycle.

RMMM — Risk Mitigation, Monitoring & Management



1. Risk Mitigation

Mitigation means reducing the chance that a risk will happen, or reducing its impact if it does happen. Actions are taken before the risk occurs.

- Example: If a key developer might leave, train a backup team member to cover that role

2. Risk Monitoring

Monitoring means continuously watching the project for signs that a risk is getting closer to happening. The team tracks warning indicators.

- Example: Regularly check employee satisfaction scores to detect if a developer is likely to leave

3. Risk Management

Management means having a prepared plan ready so that if a risk does occur, the team can respond quickly and effectively.

- Example: If a developer leaves, immediately begin the hiring process using the pre-prepared job description

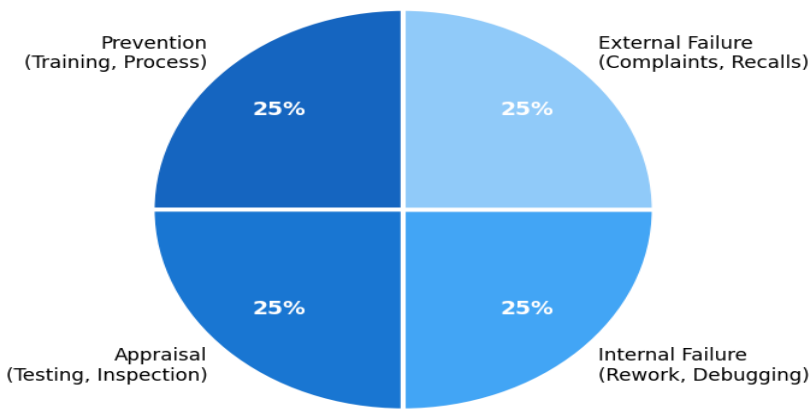
RMMM Example — Key Developer Leaves Project

RMMM Step	Action Taken
Mitigation	Cross-train other team members so they can cover the developer's tasks
Monitoring	Track developer satisfaction and monitor for early warning signs
Management	Start hiring a replacement immediately if the developer leaves

Q14. Cost of Quality (COQ) — Definition and Types

Cost of Quality (COQ) refers to all the costs a company incurs to ensure their software meets quality standards. It includes money spent on preventing problems as well as money lost when problems occur.

Cost of Quality (COQ) — Four Types



Type	Goal	Examples
Prevention Cost	Stop defects from being introduced	Staff training, process improvement, design reviews
Appraisal Cost	Evaluate and check quality during development	Testing, code inspection, quality audits
Internal Failure Cost	Fix defects found before delivery to customer	Rework, debugging, fixing failed tests
External Failure Cost	Fix defects found after delivery to customer	Customer complaints, product recalls, refunds

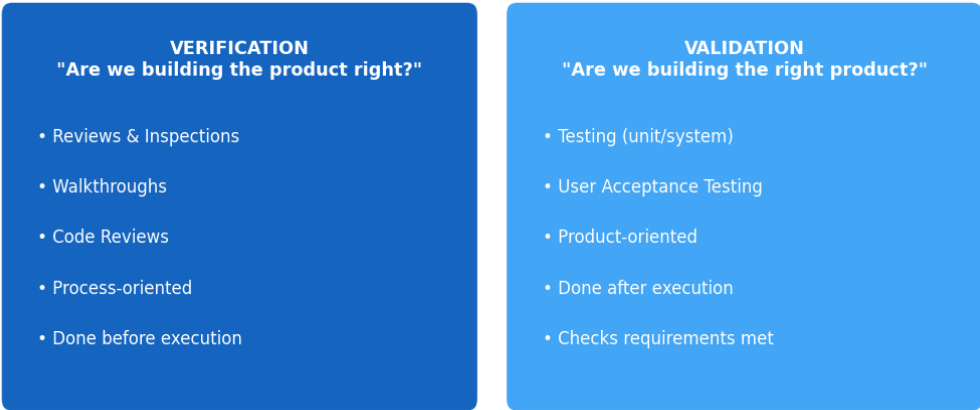
Key Insight

Prevention costs are the cheapest. The later a defect is found, the more expensive it becomes to fix. A bug found during design is much cheaper to fix than one found by a customer after release.

Q15. Software Verification and Validation

Verification and Validation (V&V) are two quality assurance activities that ensure software is correct and meets user needs. They sound similar but have different goals.

Verification vs Validation



Verification

Verification checks whether we are building the software correctly — according to the specification and design.

Key Question: "Are we building the product right?"

- Reviews — checking documents and requirements for errors
- Inspections — formally examining code or design documents
- Walkthroughs — team goes through the code step by step

Validation

Validation checks whether we are building the correct software — the software that the customer actually needs.

Key Question: "Are we building the right product?"

- Testing — running the software and checking results
- User Acceptance Testing — real users test the software

Aspect	Verification	Validation
Main Goal	Check correctness of process	Check correctness of product
Timing	Done before execution / during development	Done after execution / on working software
Focus	Are specifications followed?	Do users' requirements get satisfied?
Techniques	Reviews, Inspections, Walkthroughs	Testing, User Acceptance Testing

Q16. Black Box vs White Box Testing

Black Box Testing

Black Box Testing tests software functionality without knowing or looking at the internal code. The tester only sees inputs and outputs — like a black box where you cannot see inside.

Focus areas:

- Does the correct output appear for a given input?
- Are all requirements working as expected?
- Does the system behave correctly for valid and invalid inputs?

White Box Testing

White Box Testing tests the internal code structure and logic. The tester knows the code and examines how it works internally — like a transparent or white box where everything is visible.

Focus areas:

- Are all code paths and conditions covered?
- Are there any unreachable code sections?
- Do all loops and conditions behave correctly?

Aspect	Black Box Testing	White Box Testing
Code Knowledge	No code knowledge required	Full code knowledge required
Type of Testing	Functional testing	Structural / logic testing
Perspective	User's perspective	Developer's perspective
What is Tested?	Inputs and outputs only	Internal code, loops, conditions
Who Performs It?	QA testers	Developers

Q17. Debugging — Definition and Comparison with Testing

What is Debugging?

Debugging is the process of finding, analyzing, and fixing errors (bugs) in software code. After a test reveals that something is wrong, debugging is done to find exactly where the problem is and correct it.

Steps of Debugging

7. Identify the bug — understand what symptom or failure was observed
8. Find the root cause — trace the code to locate where the error occurred
9. Correct the code — make the fix to remove the error
10. Retest — run the test again to confirm the fix worked

Aspect	Testing	Debugging
Goal	Find defects in the software	Fix defects in the software
Done By	QA Testers	Developers

Aspect	Testing	Debugging
What it Does	Detects that errors exist	Removes the errors
Type of Activity	Quality assurance activity	Error correction activity
When it Happens	Before debugging	After testing finds a bug

Relationship Between Testing and Debugging

Testing and Debugging work together. Testing identifies that a bug exists. Debugging finds where it is and fixes it. Both are essential to producing quality software.

Q18. Software Configuration Management (SCM)

Definition

Software Configuration Management (SCM) is the process of tracking and controlling changes made to software throughout its entire lifecycle. It ensures that changes are organized, documented, and do not break existing working code.

SCM Activities

SCM Activity	Description
Configuration Identification	Identify and label all software components and versions
Version Control	Track every version of code to allow rollback if needed
Change Control	Review and approve changes before they are applied
Status Reporting	Record and report the current state of all components
Configuration Audit	Verify that software matches its specification and standards

Why is SCM Needed?

- Controls who can change software and how — prevents unauthorized edits
- Maintains version history — you can go back to any older version
- Prevents loss of source code due to accidental deletion or overwriting
- Supports team collaboration — multiple developers can work safely on the same project
- Improves software quality by reviewing changes before applying them
- Makes maintenance easier — history shows what changed, when, and why

Popular SCM Tools

- Git — most widely used version control system in the world
- GitHub / GitLab / Bitbucket — online platforms built on top of Git

Conclusion

SCM ensures that every change to software is tracked, reviewed, and applied in a controlled way. Without SCM, large team projects quickly become chaotic and unreliable.